

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 - Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 - Articulation (BL3)	Assessment:	Assessment Tool 3 - Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	A. Sathwika Reddy	Reg. No.:	192525274
Section / Batch:	2025	Date:	28/08/2026

Code of ConductI, A. Sathwika Reddy - 192525274 (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Sathwika Reddy**Instructions**

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

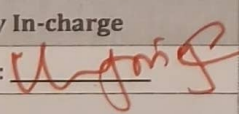
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1)

Basis	Linear Probing	Separate Chaining
Basic Idea	Coding elements are placed in next available slot in hash table.	Coding elements are stored in a linked list/chain associated with the same index.
Clustering effect	Causes primary clustering where consecutive occupied positions form long groups.	Doesn't not produce primary clustering because elements remain separate chains.
Collision handling	Searches sequentially until an empty is found.	Adds new elements to the chain at the calculated hash index.
Memory usage	Uses only hash-table array, so extra memory requirement is low.	Requires additional memory for pointers and dynamically allocated nodes.
Load factor	works efficiently only when load factor is relatively low.	Can continue working even when the load factor becomes greater than 1.
Deletion	More complicated due to deleted slots/tombstones	Easier using linked list deletion.

27

parameter	Quick sort	Merge Sort
Basic method	Divide using a pivot and partitions elements	Divides into halves and merges sorted parts.
Time - Best	$O(n \log n)$	$O(n \log n)$
Time - Avg	$O(n \log n)$	$O(n \log n)$
Time - worst	$O(n^2)$	$O(n \log n)$
Extra Space	$O(\log n)$ average of recursion	$O(n)$ for arrays, linked list can be sorted with less extra storage
Linked list	Less convenient because random access is unavailable.	Highly suitable because splitting and merging can be done efficiently.
Array s	Usually preferred because of good cache performance and low extra space.	Usually when stability and guaranteed $O(n \log n)$ are required.
Stability	usually unstable	Stable

3>

parameter	Quadratic probing	Double Hashing
collision Handling	uses quadratic jumps to find another position.	uses a second hash function to determine jump
clustering	Reduces primary clustering but may have secondary clustering	Minimizes both primary and secondary clustering
collision Resolution	moderate efficiency	generally more efficient
probe sequence	Follow a fixed quadratic pattern	Depends on 2nd hashing.
Average Search	$O(1)$	$O(1)$
worst case	$O(n)$	$O(n)$
Limitation	Secondary clustering can occur	Require additional hash function
Best use	Moderate load factors	higher/dense load factors where better collision handling needed.

4)

Parameter	Best case	Worst case
Meaning	Min time required	max time required
Input condition	most favourable arrangement	most unfavourable arrangement
Algorithm Selection	useful when favourable input is common	more imp when predictable performance required
Eg:- Quick sort	$O(n \log n)$	$O(n^2)$
Eg:- Merge sort	$O(n \log n)$	$O(n \log n)$
Reality	may not represent normal potential efficiency	critical for large or unpredicted data
Importance	useful for the understanding potential efficiency	critical for large or unpredictable datasets

5) parameters	Recursive Quick sort	Iterative Quick sort
Implementation	uses function calls recursively	uses an explicit stack / data structure
Space complexity	$O(\log n)$	$O(\log n)$
Stack usage	uses system call stack	uses manually managed stack
Ease of Implementation	Easier & more natural	more complex
Stack overflow	Higher especially with pivot selection	Lower because stack size can be controlled
control over memory	Less control over recursion	more control over stack usage
Advantage	Simple, concise and easy to implement	Avoid deep system recursion.
Limitation	Cause stack overflow	Require additional code for stack

Unqes